

Modul 1: Fondasi Jaringan Saraf, FNN, Aktivasi, dan Loss

Nama: Fiodora Alysa Juandi

NIM: 123450051

Kelas: RB

Tanggal: 20-09-2026

Berkas pengumpulan: M01_NIM.ipynb, M01_NIM.pdf, dan M01_NIM_metrics.csv.

Seluruh kode, eksperimen, grafik, dan analisis merupakan pekerjaan individual. Beri atribusi pada kode yang diadaptasi dari sumber lain.

Petunjuk

1. Ganti seluruh penanda `TODO`.
2. Jangan mengubah protokol eksperimen kecuali diminta.
3. Gunakan validation set untuk memilih model. Test set hanya untuk model final.
4. Sebelum mengumpulkan, jalankan **Restart Kernel and Run All**.
5. Pertahankan semua hasil eksperimen, termasuk hasil yang tidak sesuai hipotesis.

```
import platform
import random
import time

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import sklearn
import torch
from sklearn.datasets import make_moons
from sklearn.metrics import ConfusionMatrixDisplay, confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from torch import nn
from torch.utils.data import DataLoader, TensorDataset

NIM = 'ISI_NIM' # TODO: ganti dengan NIM lengkap
assert NIM.isdigit() and len(NIM) >= 4, 'Isi NIM dengan angka sebelum melanjutkan.'
NIM_LAST4 = int(NIM[-4:])
SEED = 1000 + NIM_LAST4
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

def seed_everything(seed: int) -> None:
    random.seed(seed)
    np.random.seed(seed)
```

```

torch.manual_seed(seed)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(seed)

seed_everything(SEED)
pd.set_option('display.precision', 4)
environment = {
    'python': platform.python_version(),
    'numpy': np.__version__,
    'pandas': pd.__version__,
    'sklearn': sklearn.__version__,
    'torch': torch.__version__,
    'device': str(DEVICE),
    'seed': SEED,
}
environment

```

A. Pre-lab - 10 poin

Jawab sebelum sesi praktikum.

1. **Parameter vs hyperparameter:** TODO
2. **Mengapa tumpukan layer affine tanpa aktivasi dapat diciutkan:** TODO
3. **Shape bobot layer 2 masukan -> 8 keluaran:** TODO
4. **Mengapa BCEWithLogitsLoss menerima logit:** TODO

Hipotesis awal

Prediksi aktivasi dan hidden size yang akan memberi validation loss terbaik. Jelaskan alasannya sebelum menjalankan eksperimen.

Jawaban: TODO

B. Neuron dan fungsi aktivasi dengan NumPy - 20 poin

```

def sigmoid_np(z: np.ndarray) -> np.ndarray:
    # Sigmoid stabil untuk nilai positif dan negatif besar
    z = np.asarray(z, dtype=float)
    out = np.empty_like(z)

    positif = z >= 0
    negatif = ~positif

    out[positif] = 1 / (1 + np.exp(-z[positif]))

    exp_z = np.exp(z[negatif])
    out[negatif] = exp_z / (1 + exp_z)

    return out

```

```

def relu_np(z: np.ndarray) -> np.ndarray:
    return np.maximum(0, z)

def leaky_relu_np(z: np.ndarray, alpha: float = 0.01) -> np.ndarray:
    return np.where(z >= 0, z, alpha * z)

def neuron_batch(X: np.ndarray, w: np.ndarray, b: float, activation):
    # Validasi shape
    if X.ndim != 2:
        raise ValueError("X harus berbentuk matriks 2 dimensi.")

    if w.ndim != 1:
        raise ValueError("w harus berupa vektor 1 dimensi.")

    if X.shape[1] != w.shape[0]:
        raise ValueError("Jumlah fitur X harus sama dengan jumlah bobot w.")

    # Pra-aktivasi
    z = X @ w + b

    # Aktivasi
    a = activation(z)

    return z, a

# Uji minimum - jangan mengubah data uji.
X_check = np.array([
    [2.0, -1.0],
    [0.0, 3.0],
    [-2.0, 1.0]
])

w_check = np.array([0.5, -0.5])

z_check, a_check = neuron_batch(
    X_check,
    w_check,
    0.25,
    sigmoid_np
)

assert z_check.shape == (3,)
assert a_check.shape == (3,)
assert np.all(np.isfinite(a_check))
assert np.all((a_check > 0) & (a_check < 1))

```

```
pd.DataFrame({
    'z': z_check,
    'sigmoid(z)': a_check
})
```

```
-----
NameError                                Traceback (most recent call
last)
```

```
Cell In[5], line 1
```

```
----> 1 def sigmoid_np(z: np.ndarray) -> np.ndarray:
      2     # Sigmoid stabil untuk nilai positif dan negatif besar
      3     z = np.asarray(z, dtype=float)
      4     out = np.empty_like(z)
```

```
NameError: name 'np' is not defined
```

Interpretasi aktivasi

- Aktivasi yang berpusat di nol: TODO
- Aktivasi yang tepat nol untuk masukan negatif: TODO
- Aktivasi yang mudah jenuh: TODO
- Mengapa aktivasi linear tidak cukup untuk `make_moons`: TODO

Analisis Fungsi Aktivasi

1. Fungsi yang berpusat di nol adalah **tanh**, karena keluarannya berada pada rentang (-1, 1) dan memiliki nilai 0 ketika input bernilai 0.
2. Fungsi yang menghasilkan tepat nol untuk seluruh input negatif adalah **ReLU**, karena ReLU didefinisikan sebagai $\max(0, z)$.
3. **Sigmoid** paling mudah mengalami saturasi. Pada input yang sangat negatif outputnya mendekati 0, sedangkan pada input yang sangat positif outputnya mendekati 1 sehingga perubahan output menjadi sangat kecil.
4. Aktivasi linear tidak cukup untuk pola dua bulan sabit karena data `make_moons` memiliki batas kelas yang nonlinier. Tanpa aktivasi nonlinier, beberapa layer affine tetap ekuivalen dengan satu transformasi affine sehingga tidak mampu membentuk batas keputusan yang melengkung.

C. Forward pass manual - bagian dari 20 poin pemetaan

Gunakan arsitektur $2 \rightarrow 2 \rightarrow 1$, ReLU pada hidden layer, dan target $y=1$. Jangan mengubah bobot.

```
import numpy as np
```

```
x_manual = np.array([[2.0, -1.0]], dtype=np.float32)
```

```

W1 = np.array([[0.5, -0.5], [1.0, 1.0]], dtype=np.float32)
b1 = np.array([0.0, 0.0], dtype=np.float32)
W2 = np.array([[2.0, -1.0]], dtype=np.float32)
b2 = np.array([0.5], dtype=np.float32)
y_manual = np.array([[1.0]], dtype=np.float32)

z1_np = x_manual @ W1.T + b1
h_np = np.maximum(0.0, z1_np)
logit_np = h_np @ W2.T + b2
prob_np = 1.0 / (1.0 + np.exp(-logit_np))
bce_np = -(y_manual * np.log(prob_np) + (1.0 - y_manual) * np.log(1.0 - prob_np))

parameter_count_manual = W1.size + b1.size + W2.size + b2.size

print('z1 =', z1_np)
print('h =', h_np)
print('logit =', logit_np)
print('probability =', prob_np)
print('BCE =', bce_np)
print('parameter count =', parameter_count_manual)

```

```

-----
-----
NameError                                Traceback (most recent call
last)
Cell In[9], line 1
----> 1 x_manual = np.array([[2.0, -1.0]], dtype=np.float32)
      2 W1 = np.array([[0.5, -0.5], [1.0, 1.0]], dtype=np.float32)
      3 b1 = np.array([0.0, 0.0], dtype=np.float32)
      4 W2 = np.array([[2.0, -1.0]], dtype=np.float32)

NameError: name 'np' is not defined

```

Tabel shape dan nilai

Tensor	Shape	Nilai
x	TODO	TODO
$W^{(1)}$	TODO	tersedia pada kode
$z^{(1)}$	TODO	TODO
h	TODO	TODO
logit	TODO	TODO
probabilitas	TODO	TODO
BCE	skalar	TODO

Jumlah parameter dan perhitungannya: TODO

D. Pencocokan NumPy dan PyTorch - 20 poin

Salin bobot ke `nn.Sequential`. Jangan menambahkan sigmoid ke model karena loss menerima logit.

```
manual_model = nn.Sequential(
    nn.Linear(2, 2),
    nn.ReLU(),
    nn.Linear(2, 1),
)

# TODO: salin W1, b1, W2, b2 menggunakan torch.no_grad().

# TODO: hitung torch_logit, torch_probability, dan torch_loss.
torch_logit = None
torch_probability = None
torch_loss = None

# TODO: aktifkan dan lengkapi pemeriksaan berikut.
# assert np.max(np.abs(torch_logit.detach().numpy() - logit_np)) < 1e-6
# assert abs(torch_loss.item() - float(bce_np)) < 1e-6
# assert sum(p.numel() for p in manual_model.parameters()) == parameter_count_manual
```

Checkpoint menit ke-85

Tunjukkan kepada asisten:

- plot empat fungsi aktivasi;
- tabel shape dan parameter; dan
- selisih forward NumPy-PyTorch kurang dari 10^{-6} .

Status/verifikasi asisten: TODO

E. Dataset dan protokol eksperimen

Kode split dan standardisasi diberikan agar fokus tetap pada FNN. Jangan menggunakan test set sampai model final dipilih.

```
X, y = make_moons(n_samples=600, noise=0.22, random_state=SEED)
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.40, stratify=y, random_state=SEED
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, stratify=y_temp, random_state=SEED
)
scaler = StandardScaler().fit(X_train)
X_train_s = scaler.transform(X_train).astype(np.float32)
```

```

X_val_s = scaler.transform(X_val).astype(np.float32)
X_test_s = scaler.transform(X_test).astype(np.float32)
y_train_f = y_train.astype(np.float32).reshape(-1, 1)
y_val_f = y_val.astype(np.float32).reshape(-1, 1)
y_test_f = y_test.astype(np.float32).reshape(-1, 1)

split_summary = pd.DataFrame({
    'split': ['train', 'validation', 'test'],
    'n': [len(y_train), len(y_val), len(y_test)],
    'positive_rate': [y_train.mean(), y_val.mean(), y_test.mean()],
})
display(split_summary)

fig, ax = plt.subplots(figsize=(6, 4.5))
ax.scatter(X_train_s[:, 0], X_train_s[:, 1], c=y_train,
           cmap='coolwarm', s=24, alpha=0.75)
ax.set(title='Train Set make_moons', xlabel='fitur 1', ylabel='fitur 2')
ax.grid(alpha=0.2)
plt.show()

```

Pemeriksaan anti-leakage

Jelaskan mengapa scaler hanya di-fit pada train set dan bukan seluruh data.

Jawaban: TODO

F. Model dan utilitas training

Lengkapi pemilihan aktivasi dan model. Training loop diberikan; mekanismenya dibahas pada Modul 2.

```

def activation_layer(name: str) -> nn.Module:
    # TODO: kembalikan objek nn.ReLU(), nn.Tanh(), atau nn.Sigmoid().
    raise NotImplementedError

def build_model(hidden_dim: int, activation: str, seed: int = SEED):
    seed_everything(seed)
    # TODO: bangun arsitektur 2 -> hidden_dim -> 1 tanpa sigmoid
    output.
    raise NotImplementedError

def count_parameters(model: nn.Module) -> int:
    return sum(parameter.numel() for parameter in model.parameters())

# TODO: buat model 2 -> 8 -> 1 dan buktikan jumlah parameternya 4h+1 = 33.
# model_check = build_model(8, 'relu')
# assert count_parameters(model_check) == 33

```

```

def as_tensor(array):
    return torch.as_tensor(array, dtype=torch.float32)

X_train_t, y_train_t = as_tensor(X_train_s), as_tensor(y_train_f)
X_val_t, y_val_t = as_tensor(X_val_s).to(DEVICE),
as_tensor(y_val_f).to(DEVICE)
X_test_t, y_test_t = as_tensor(X_test_s).to(DEVICE),
as_tensor(y_test_f).to(DEVICE)

def evaluate(model, X_tensor, y_tensor):
    model.eval()
    with torch.no_grad():
        logits = model(X_tensor)
        loss = nn.functional.binary_cross_entropy_with_logits(logits,
y_tensor).item()
        probabilities = torch.sigmoid(logits)
        predictions = (probabilities >= 0.5).to(torch.int64)
        accuracy = (predictions ==
y_tensor.to(torch.int64)).float().mean().item()
    return {
        'loss': loss,
        'accuracy': accuracy,
        'probabilities': probabilities.cpu().numpy().ravel(),
        'predictions': predictions.cpu().numpy().ravel(),
    }

def train_model(model, epochs=200, learning_rate=0.05, batch_size=32,
seed=SEED):
    generator = torch.Generator().manual_seed(seed)
    loader = DataLoader(
        TensorDataset(X_train_t, y_train_t),
        batch_size=batch_size, shuffle=True, generator=generator,
    )
    optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
    criterion = nn.BCEWithLogitsLoss()
    history = []
    start = time.perf_counter()
    for epoch in range(1, epochs + 1):
        model.train()
        loss_sum = 0.0
        for X_batch, y_batch in loader:
            X_batch, y_batch = X_batch.to(DEVICE), y_batch.to(DEVICE)
            optimizer.zero_grad()
            logits = model(X_batch)
            loss = criterion(logits, y_batch)
            loss.backward()
            optimizer.step()
            loss_sum += loss.item() * len(X_batch)
        val_metrics = evaluate(model, X_val_t, y_val_t)
        history.append({

```

```

        'epoch': epoch,
        'train_loss': loss_sum / len(X_train_t),
        'val_loss': val_metrics['loss'],
        'val_accuracy': val_metrics['accuracy'],
    })
    return pd.DataFrame(history), time.perf_counter() - start

```

G. Baseline dan latihan kelas

Baseline: ReLU, hidden size 8, SGD, learning rate 0.05, batch size 32, dan 200 epoch.

Latihan individual: digit terakhir NIM 0-4 memakai tanh; 5-9 memakai sigmoid. Prediksi hasil sebelum menjalankan.

Prediksi: TODO

```

# TODO: latih baseline, tampilkan kurva train/validation loss, dan
# catat metrik.
# TODO: latih satu variasi individual dengan komponen lain tetap.
raise NotImplementedError('Selesaikan baseline dan latihan
individual.')

```

H. Tugas individual: enam eksperimen - 20 poin

Jalankan kombinasi aktivasi {relu, tanh, sigmoid} dan hidden size {4, 16}. Gunakan validation loss untuk memilih model.

```

ACTIVATIONS = ['relu', 'tanh', 'sigmoid']
HIDDEN_SIZES = [4, 16]
EPOCHS = 200
LEARNING_RATE = 0.05
BATCH_SIZE = 32

experiment_rows = []
trained_models = {}
histories = {}

# TODO: untuk setiap kombinasi:
# 1. buat model baru dengan seed yang sama;
# 2. latih dengan anggaran tetap;
# 3. evaluasi train dan validation;
# 4. simpan model/history;
# 5. tambahkan satu dictionary metrik ke experiment_rows.

# results =
pd.DataFrame(experiment_rows).sort_values('val_loss').reset_index(drop=True)
# assert len(results) == 6
# display(results)

```

```
# results.to_csv(f'M01_{NIM_LAST4:04d}_metrics.csv', index=False)
raise NotImplementedError('Jalankan enam eksperimen dan simpan
metrics.csv.')
```

I. Evaluasi model final - 15 poin

Pilih satu model dengan validation loss terendah. Baru setelah itu evaluasi test set satu kali.

```
def plot_decision_boundary(model, X_values, y_values, title):
    # TODO: buat mesh, prediksi probabilitas, contourf, batas 0.5, dan
    scatter data.
    raise NotImplementedError

# TODO: ambil run_id terbaik dari results.
# TODO: hitung test loss dan test accuracy satu kali.
# TODO: tampilkan confusion matrix dan decision boundary model final.
raise NotImplementedError('Evaluasi model final dan buat
visualisasi.')
```

J. Analisis dan refleksi - 10 poin

Jawab dengan merujuk angka atau grafik.

1. Apakah hidden size lebih besar selalu memperbaiki validation loss? **TODO**
2. Aktivasi mana yang paling stabil pada dua hidden size? **TODO**
3. Adakah konfigurasi dengan accuracy serupa tetapi BCE berbeda? Mengapa? **TODO**
4. Di bagian mana decision boundary paling tidak pasti? **TODO**
5. Apa satu keterbatasan eksperimen ini? **TODO**
6. Apakah hasil sesuai hipotesis awal? **TODO**

Checklist pengumpulan - 5 poin reproduksibilitas

- ☐ Identitas, seed, versi library, dan device tercantum.
- ☐ Forward NumPy-PyTorch memiliki selisih kurang dari 10^{-6} .
- ☐ Enam eksperimen tercatat pada notebook dan CSV.
- ☐ Test set hanya dipakai untuk model final.
- ☐ Semua grafik memiliki judul, label sumbu, dan legenda/caption.
- ☐ Notebook lolos Restart Kernel and Run All.
- ☐ Tidak ada path absolut atau data pribadi.
- ☐ Sumber eksternal telah diberi atribusi.

Pernyataan orisinalitas

Saya menyatakan bahwa kode, eksperimen, visualisasi, dan analisis pada notebook ini merupakan pekerjaan individual saya.

Nama dan tanggal: TODO